

Lecture 7: Frontend Full-stack Application Development

Mobile and Multi-platform Application/XPC-MMA

Michal Harvanek
`michal.harvanek@vut.cz`

Department of Radio Electronics, Brno University of Technology, Czechia

2024/25

Contents:

- 1 Full stack application (PART 2)
- 2 Flutter Frontend

- 3 Task
- 4 Further reading

Full stack application (PART 2)

- Django backend with features:
 - Django application
 - Built API with REST framework
 - Prepared endpoints for Notes application

Project `urls.py` including application's urls endpoints

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.getRoutes),
    path('notes/', views.getNotes),
    path('notes/create/', views.createNote),
    path('notes/<str:pk>/update', views.updateNote),
    path('notes/<str:pk>/delete', views.deleteNote),
    path('notes/<str:pk>', views.getNote),
]
```

■ What are CORS headers?

- CORS (Cross-Origin Resource Sharing) allows or restricts requests from different origin domains when accessing Django API.
- Default browser policy is to restrict fetching data from different local hosts (ie, `http://localhost:3000` and `http://localhost:8000`).
- Occurs typically, when your front-end application running on one port and the back-end on the other port.
- The reason of restrictions is CORS settings help prevent unauthorized domains from accessing your API.

- Install the required package.

Install django-cors-headers

```
pip install django-cors-headers
```

- Add 'corsheaders' to the list of installed applications.

Add corsheaders to `INSTALLED_APPS` in `settings.py`

```
INSTALLED_APPS = [  
    ...  
    'corsheaders',  
]
```

- Add CORS middleware at the top of the middleware list.

Add CORS middleware in settings.py

```
MIDDLEWARE = [  
    'corsheaders.middleware.CorsMiddleware',  
    'django.middleware.common.CommonMiddleware',  
    ...  
]
```

- Define allowed origins for CORS.

Example of configuration of allowed origins in settings.py

```
CORS_ALLOWED_ORIGINS = [  
    "http://localhost:3000",  
    "https://myfrontend.com",  
]
```

- Allow all origins

Example of configuration of allowed all origins in settings.py

```
CORS_ALLOW_ALL_ORIGINS = True
```

Flutter Frontend

Generate Flutter project from terminal

- Create folder flutter in your django app and create flutter project using command line.
- Install the required package.

Install django-cors-headers

```
flutter create noteapp
```

- The file structure should look like:

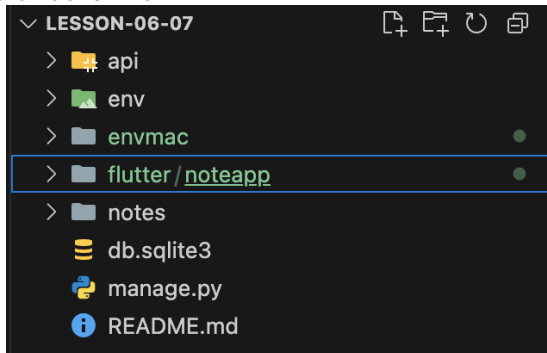


Figure: Folder organisation.

Use http package to handle http requests

- Find the **http** package at <https://pub.dev>.
- Add http package to your 'pubspec.yaml' dependencies

Add http packages to pubspec.yaml

```
dependencies:  
  flutter:  
    sdk: flutter  
  cupertino_icons: ^1.0.8  
  http: ^1.3.0
```

- run command pub get:

Run pub get to get added dependencies

```
flutter pub get
```

- Create CreatePage - for creating Note (create.dart)
- Create UpdatePage - for updating Note(update.dart)
- Create Note page - for fetch and view note from django backend. (note.dart)
- Create urls file which translates your endpoints to dart code. (urls.dart)
- main.dart Create http client and method for retrieving data from server.
- TIPS:
 - Use `ctrl + .` (`cmd + .`) to show flutter context menu.
 - Use `F1` remove all comments to delete comments from generated files.
 - Use `Ctrl` (`cmd`) + `s` should autoformat your dart code automatically.

Task

Create backend for Flutter Note application using python Django backend.

- Install CORS headers, check all endpoints in backend (1b).
- Make frontend Note application (9b).

Further reading

- [1] TechVidvan Team (2025).

Django Project Structure and File Structure. Online; accessed March 2025.

<https://techvidvan.com/tutorials/django-project-structure-layout/>.

- [2] YouTube (2025).

Django Tutorial Video by Denis Ivy. Online; accessed March 2025.

<https://www.youtube.com/watch?v=VnzChBw70g>.